The Complexity of Credit Risk Data Engineering

Context: Home Credit Dataset · Kaggle 1st Place Insights

March 11, 2026

1 The Real Challenge: Heterogeneous, Time-Traveling Data

As the 1st Place Kaggle Winner noted, credit underwriting is one of the most complex domains for machine learning because the data is:

- **Heterogeneous:** It mixes static demographics, changing incomes, point-in-time bureau snapshots, and high-frequency transaction logs.
- **Collected over different time frames:** A borrower's income might have been recorded 2 years ago, their bureau score 1 month ago, and their last late payment 3 days ago.
- **Subject to changing collection environments:** A bank might switch from asking "Monthly Income" to "Annual Income" in the middle of a database's life, silently breaking standard logic.

1.1 The Target Variable Trap

Defining the target (`1` = Default, `0` = Paid) is arguably the hardest part of credit ML.

- Do you define a default as 30 days past due (DPD), 60 DPD, or 90 DPD?
- If someone is 30 days late but then pays it all back (a "cure"), are they a `1` or a `0`?
- What is the performance window? Do we wait 12 months to label a loan, or 24 months?

The Home Credit dataset is celebrated because they solved these domain complexities *before* releasing the data, making it "leak-free" (meaning you cannot accidentally use future data to predict the past).

2 Smart Features vs. Base Algorithms

The winning Kaggle philosophy relies on two pillars:

1. **Good set of smart features** (Domain Knowledge)
2. **Diverse base algorithms** (Ensembling LightGBM, XGBoost, CatBoost, Neural Nets)

While anyone can run a LightGBM model, the magic is in the feature engineering. The winner used **∼700 engineered features**, primarily generated by aggregating many-to-one tables.

2.1 The "Many-to-One" Aggregation Strategy

For every single borrower in `application_train.csv` (1 row per user), there are dozens of rows in other tables:

- 5 previous applications in `previous_application.csv`
- 30 historical credit accounts in `bureau.csv`
- 120 past payments in `installments_payments.csv`

You cannot feed 120 rows into a standard ML model to predict 1 outcome. You must flatten them into "smart aggregates."

2.1.1 Examples of Smart Aggregates (The 700 Features):

- **Averages:** What is their *average* past due amount across all previous loans?
- **Max/Mins:** What is the *maximum* number of days they were ever late?
- **Recency Weighted:** How many times were they late *in the last 3 months* vs. *in the last 2 years*? (Recent behavior is always more predictive).
- **Ratios:** (Total past payments made) ÷ (Total past payments required).

3 Data Cleaning: The Unseen Foundation

Before you can aggregate, you have to clean. Credit data is notorious for silent errors.

3.1 The Sentinel Value Problem

In our notebook, we saw:

```
frame["DAYS_EMPLOYED_ANOM"] = (frame["DAYS_EMPLOYED"] == 365243).astype(int)
frame.loc[frame["DAYS_EMPLOYED"] == 365243, "DAYS_EMPLOYED"] = np.nan
```

- A value of `365243` days employed equals **1,000 years**.
- This was not an immortal applicant; it was Home Credit's system using `365243` as a "sentinel value" to mean "Unemployed."
- If you fed this directly to a model, the algorithm would think this group is extremely employed and stable, completely reversing the actual risk profile (unemployed people are highly risky).

3.2 Safe Math in Finance

```
def safe_divide(a, b):
    if isinstance(b, pd.Series):
        b = b.replace({0: np.nan})
    return a / b
```

In credit data, dividing by zero happens constantly (e.g., trying to find `Income ÷ Debt` when they have $0 debt). A standard Python `ZeroDivisionError` will crash your entire pipeline. `safe_divide` forces these to `NaN` (Missing), which Gradient Boosting models handle natively.

4 The Goto Financial / Fintech Context

For a modern fintech dealing with thin-file borrowers, the aggregation strategy shifts from formal banking tables to behavioral logs:

1. **Velocity Aggregates:** Instead of `bureau.csv`, you aggregate GoPay logs. *How many times did this user transfer money out immediately after receiving a transfer in?* (High velocity = bridging loans/distress).

2. **Time-of-Day Features:** *What percentage of their transactions happen between 12 AM and 4 AM?* (Late night high-frequency transactions often correlate with risky behavior like online gambling).

3. **App Telemetry:** *How long did they spend reading the terms and conditions? Did they copy-paste their ID number or type it out?* (Typing it from memory often indicates fraud rings).

The 1st place Kaggle lesson remains true: the algorithm is a commodity. The competitive moat is how creatively you can aggregate thousands of messy historical data points into a single, clean user profile.